

The Linear Model (LM)

T. Florian Jaeger

2026-09-06

Readings and assignments in *preparation of this class*

In preparation for class, please read *Regression and other stories (ROS)*, Ch. 3.3, Ch. 6-6.3, Ch. 7.1. Optionally, you can read 6.4-6.5 and 7.2. The final part (7.2) is a great way to consolidate your understanding by following along the simulation code they provide. This text is quite intuitive and assumes comparatively little background. **You should plan for ~3h of reading and summarizing your questions in preparation for this week of the class.**

About this document

This PDF is generated from an Quarto markdown document (using the `tufte` package for styling). If you're interested in understanding the code that generated the examples presented below, you can [clone the repo](#). Both the PDF and the R markdown that the PDF is generated from are in the `scripts/` folder. You can go through the document line by line, or make changes and 'knit' a new PDF with the press of a button.

The linear model (LM)

LMS (aka linear regression) can be thought of as an example of a much larger family of regression models that describe an *outcome* variable y (aka the *dependent* variable) as the function g of one or more *predictor* variables $x_1, \dots, x_k$ (aka *independent* variables), $y = g(x_1, \dots, x_k)$. In this sense, the LM is a—very constrained—*predictive model*.¹ Specifically, the LM assumes that y is a draw from

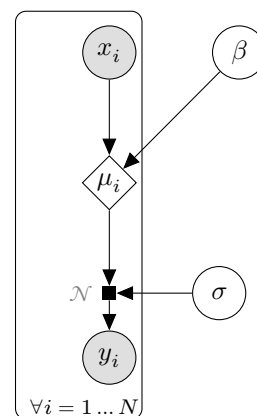


Figure 1: A graphical model visualization of the linear model, shown here in case you find it helpful. Shaded circles are observable variables. White circles are latent variables. These are the variables that constitute the degrees of freedom of the model, the parameters that are fit to the data. Diamond shapes indicate variables that are determined by other variables. The index $i = 1, \dots, N$ refers identifies the observations (usually, the rows) in your data. Note that σ , unlike μ_i is assumed to be constant across observations. This is the assumption of the homogeneity of variance, which you might have heard of if you're familiar with t -tests or ANOVA.

¹ While the LM is a specific case of a much wider class of models, it is already a rather general model. For example, t -tests and analysis of variances (ANOVA) are special cases of LMs. We'll return to this point in a later lecture.

a Normal distribution $\mathcal{N}$ with constant standard deviation σ and a mean μ that is a *linear combination* (the weighted sum) of all the predictors included in the model:

$$\begin{aligned} \mathbf{y} &\sim \mathcal{N}(\mu, \sigma_{resid}) & (1) \\ \mathbf{E}[y] &= \beta_1 \mathbf{x}_1 + \dots \beta_k \mathbf{x}_k & (2) \\ &= X\beta & (3) \end{aligned}$$

Here, y is a column vector of outcome observations of length n (e.g., a column in your data table), X is called the $n \times k$ *model (or design) matrix* (e.g., in the simplest case, a number of k columns in your data table) and β is a vector of weight values of length k (aka coefficients)—one for each of the k predictors in the model. $\mathbf{E}[y]$ refers to the *expected value* of y , which is the mean of the Normal distribution from which y is assumed to be drawn.

In the literature, LMs are often written in a number of different ways. For instance, one might describe the linear model for each observation i of the outcome y_i and all predictors $x_{1,i}, \dots, x_{k,i}$:²

² ϵ_i is the *residual* for observation i (= observed y_i - predicted $\hat{y}_i$).

$$y_i = \beta_1 x_{1,i} + \dots + \beta_k x_{k,i} + \epsilon_i, \epsilon_i \sim \text{Normal}(0, \sigma_{resid}) \quad (4)$$

or in vector or matrix form, as in James et al. (2013, p. 63):

$$\mathbf{y} = \beta_1 \mathbf{x}_1 + \dots + \beta_k \mathbf{x}_k + \epsilon, \epsilon \sim \text{Normal}(0, \sigma_{resid}) \quad (5)$$

$$= X\beta + \epsilon, \epsilon \sim \text{Normal}(0, \sigma_{resid}) \quad (6)$$

or combining equations (1) and (3) from above, as in Gelman & Hill (2007, p. 38):

$$\mathbf{y} = \text{Normal}(X\beta, \sigma_{resid}) \quad (7)$$

Any of these formulations are equivalent, and all are used in the literature. I'm mentioning them here only since it can be confusing as a learner to see these different notations. There is no need to memorize them. Finally, a common—but arbitrary—convention is to use a different symbol for one particular predictor, the *intercept* α :

$$y \sim \mathcal{N}(\mu, \sigma) \quad (8)$$

$$\mu = \alpha + \beta_1 x_1 + \dots \beta_k x_k \quad (9)$$

$$= \alpha x_0 + \beta_1 x_1 + \dots \beta_k x_k \quad (10)$$

Here, x_0 is assumed to be the constant 1, i.e., $x_0 = 1$. This convention singles out α , which helps to realize that it has an intuitive interpretation: it is the predicted value of the outcome y when all $x_1 = \dots = x_k = 0$.³

Assumptions of the LM

As a theoretical model the LM makes several assumptions (cf. Gelman & Hill, 2007, p. 45). The first two assumptions are shared with many/most other statistical models we use (to be precise, some models don't strictly make these assumptions, but they do correct for their violations):

- **Independence of observations/errors.** Each observation y is assumed to be drawn independent of all other observations.
- **Validity/exhaustivity.**

The other assumptions are more specific to the LM, though the last two of these are shared with generalized linear model (GLM) and generalized linear mixed-effect model (GLMM) that we'll cover in later lectures:⁴

- **Normality of errors.** y is drawn from a Normal distribution $\mathcal{N}$ (with mean $\mu = X\beta$), and thus is assumed to be able to take any real value from $-\infty$ to $+\infty$. Equivalently, we can say that the *residuals* (or prediction errors) are drawn from a Normal distribution (with mean 0).
- **Equal variance of errors.** While $E[y] = \mu$ is assumed to vary with the predictors X , the residual standard deviation around that mean σ is assumed to be independent of X , and thus constant and identical across all observations.
- **Additivity of effects.** The *effects* of predictors are assumed to be additive: $E[y]$ is assumed to be the weighted *sum* of the predictors.
- **Linearity of each effects.** y is linear in the β s. That is, the *effects* of predictors are assumed to be linear.

Many, if not most, of these assumptions are actually violated when we apply the model. To some extent linear regression is robust to such violations, depending on the specific assumption. For example,

³ Depending on how we *code* the predictors $x_1 = \dots = x_k = 0$, the intercept can have even conceptually relevant interpretations. For example, if all predictors are centered (so that their means are 0), then the intercept is the predicted sample mean of all y_i , $E[E[y_i]] = E[y]$. We'll learn more about predictor coding in a later lecture.

⁴ The final two assumptions sometimes confuse learners, and we'll return to them in later lectures. For now, I note that it can be helpful to distinguish between **input variables** and the **predictors** we derive from them. For example, in the case study presented below, one *input variable* is *VOT*. But we could include both *VOT* and *VOT*² as separate *predictors* in the LM. This would model both linear and quadratic effects of *VOT*, both of which are assumed to have linear effects on the outcome y . Similarly, we can include interactions of the input variable *VOT* and some other predictor in the model, which will add their product as a predictor to the model. That is, we can very much use an LM to ask whether two variables interact. The additivity assumption just means that the effect of the interaction predictor is assumed to be added to the effects of the other predictors in the model.

instead of equality of variance it is often sufficient that variances are similar (*homogeneity* of variance) or that the variance-covariance matrix does not exhibit *heteroscedasticity*.⁵ **But violations of the independence, normality, or homoscedasticity assumptions can make a model invalid, and the statistical inferences based on it invalid** (e.g., by inflating Type I and II errors). We return to these points in later classes.

⁵ From Ancient Greek *hetero*—“different” and *skedasis*—“dispersion”. Homoscedasticity extends the notion of homogeneity of variance to the covariance between variables

Use of the LM in our field(s)

While the LM can be used as a predictive model—i.e., to make predictions about unseen data—this is not the primary use of the LM in our fields. Rather, we tend to fit an LM (or other regression models) to the sample of data we have collected, and then conduct statistical inferences about the *effects* of one or more of the model’s predictors based on the best-fitting parameterization of that model. Best-fitting parameterization refers to the estimates of β s (and the residual standard deviation σ , though we don’t tend to be interested in this parameter) that make the LM deliver the best predictions about the observed values of y . We’ll briefly cover what “best” means in this context in the next section, and we’ll return to it in more depth later in the semester.

When we employ an LM during data analysis, we are making the assumption that this model describes the relation between the predictors and the outcome in the population we seek to study. This also means that all of our conclusions are contingent on these assumptions, including in particular the assumption that we have considered all relevant variables (or have otherwise ruled out that other effects can confound our analysis).

Geometric intuitions

Generally speaking, an LM with k predictors (plus the intercept) can be thought of as defining a hyperplane in a $k + 1$ dimensional space, where the additional dimension is the outcome. We’ll cover this part in more depth in class since it’s easier to appreciate on a whiteboard. A great introduction with effective visualizations is provided in James et al. (2013, Ch 3) *Introduction to statistical learning*.

For instance, for a single predictor, the LM defines a line that describes the predicted outcome as a function of the single predictor. For two predictors, the LM defines a plane in the 3D space defined by the two predictors and the outcome, etc. (for some examples, see Figure 2).

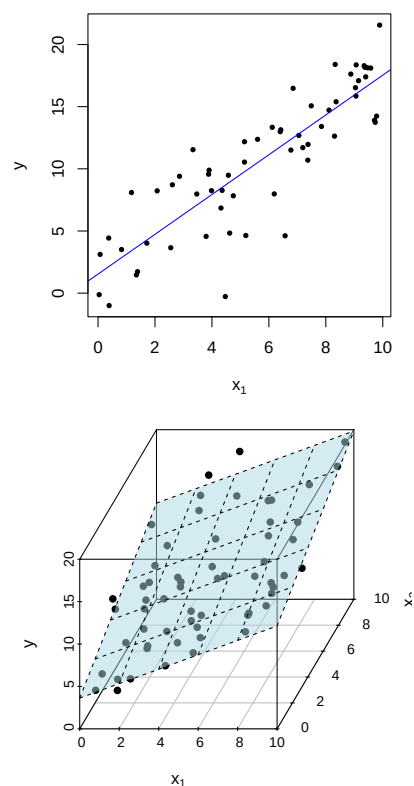


Figure 2: Examples of an LM with one continuous predictor (top; the line) and two continuous predictors (bottom; the plane).

What makes a line fit well? Residuals, RSS, and likelihood

When we look at Figure 2, it seems intuitive that the line/hyperplane provides a relatively good fit to the points. But what does that mean and how can we quantify *goodness-of-fit*? Later in the semester, we'll learn different approaches to this question and how they relate to each other. For now, here's a brief primer in case you're curious, but don't worry if it doesn't yet all make sense.

We illustrate this here for the single-predictor case, since it is the easiest to draw. There are two related but distinct ideas: (1) a line fits better the smaller its (squared) residuals are, and (2) a line fits better the higher the *likelihood* of the observed data is under the normal-error assumption of the LM. It turns out that, for LMs, the two approaches are identical and yield the same best-fitting model.

Residuals and the residual sum of squares (RSS). For any candidate line $\hat{y}_i = \hat{\alpha} + \hat{\beta}x_i$, the residual for observation i is the vertical distance between the predicted and the actual outcome, $y_i - \hat{y}_i$ (the red lines below). Squaring and summing these residuals gives the residual sum of squares, $RSS = \sum_i (y_i - \hat{y}_i)^2$, an intuitive measure of how 'off' the model's predictions are. The three panels below show the same data with three different hypothetical lines: the actual best (least-squares) fit, a line with the same slope but shifted down, and a line with the (best-fitting) intercept but a different slope. Both of the latter two lines fit clearly worse (larger red residuals, larger RSS) than the best-fitting line. The best-fitting line is the one that minimizes the RSS.

Likelihood under the normal-error assumption. Recall that the LM assumes $y_i \sim \mathcal{N}(\hat{\alpha} + \hat{\beta}x_i, \hat{\sigma})$: for every value of x , the model places an *identical-variance* normal distribution around the predicted value $\hat{y}$. The fourth panel below visualizes this by drawing a few of these normal distributions (in red), rotated so that they sit vertically on top of the fit line. For a few example points, the annotated value is the *log-density* of that point's observed y_i under its corresponding normal distribution, $\log p(y_i \mid \hat{\alpha}, \hat{\beta})$ —points close to the line fall in the peak of the density (higher, i.e., less negative log-density) than points far from the line. Summing the log-densities across all observations gives the (log-)likelihood of the data under the fitted line, $\Lambda = \sum_i \log p(y_i \mid \hat{\alpha}, \hat{\beta})$. It turns out that, for the LM, choosing $\hat{\alpha}, \hat{\beta}$ to maximize Λ —maximizing the model's likelihood—is the *same* as minimizing the RSS—minimizing the summed (squared) prediction error!

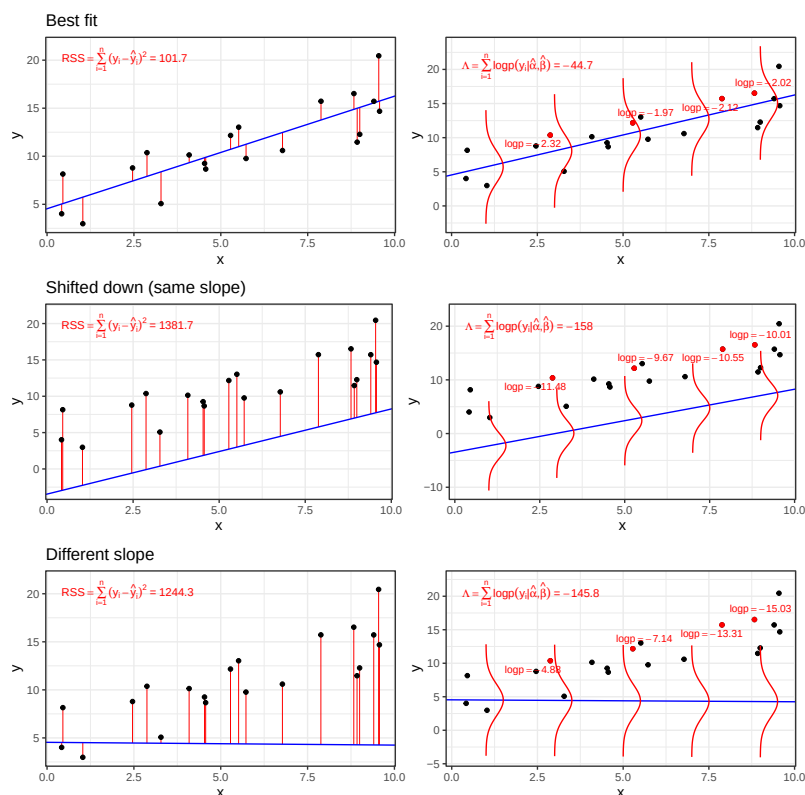


Figure 3: Three hypothetical lines for the same data (rows), with the resulting residuals and RSS (left column) and the corresponding normal-likelihood visualization (right column). Points colored in red (right column) are the ones for which the log-density is annotated. The best-fitting line (top row, boldface summary) has the smallest RSS and the highest log-likelihood of the three.

General workflow

Here's the general workflow I'd recommend for an LM analysis, following adequate data checks, variable typing (e.g., making sure that variables that are factors indeed are factors, etc.) and filtering (e.g., removal of practice and/or catch trials, outlier exclusion, etc.). Some of these steps won't yet make sense since they refer to concepts we have not yet covered. I mention them here so that you can use this workflow section as a general recipe to look up later.

- Get data into table format, with each observation of your data

corresponding to one row of the table and separate columns for the outcome and each of the input variables. What counts as an observation depends on the question you intend to ask.

- Decide whether to transform your continuous input variables, e.g., by centering and/or standardizing them. Decide how to code the factors in your data, so that each of the factor input variables can be translated into one or more numerical predictors. We'll cover how to code categorical predictors in a separate lecture. For now, I'll note that most statistics packages will quietly apply default coding if you do not specify it yourself. Since the interpretation of the regression output depends on the specific predictor coding, this is something that you should always keep in mind.
- Define the regression formula $y \sim 1 + x_1 + \dots + x_k$, **where 1 stands for the constant 1**. Together with the data table and information about the factor coding (which is stored as an attribute to the factor column in the data.frame in R and in many similar programs), this formula contains all the information that is required for a function intended to fit the model to the data.

A call to the LM-fitting function typically first implicitly creates the model matrix X . **Looking at the model matrix corresponding to an LM or other model can be a very effective way to learn to understand formula syntax in whatever language you're using.** By default, this model matrix will contain the constant 1 as the first column, followed by one column per *predictor*. Note that this can be more columns than the number of input variables. For example, factor input variables with k distinct levels will result in $k - 1$ distinct predictors, and thus $k - 1$ distinct columns in the model matrix. Similarly, interactions in the formula add additional columns to the model matrix (more on that in a later lecture). And there are a number of other common formula abbreviations that will expand into multiple columns in the design matrix. If you're not sure what a formula does, I recommend simply looking at the model matrix it creates (in R, `model.matrix(lm(...))`):

```
# A tibble: 15 x 6
  `(Intercept)` ConditionShift10 ConditionShift40 VOT `ConditionShift10:VOT` `ConditionShift40:VOT`
    <dbl>          <dbl>          <dbl> <dbl>          <dbl>          <dbl>
1         1         1         0    15         15         0
2         1         0         1    30         0         30
3         1         0         0     0         0         0
4         1         1         0    45         45         0
5         1         0         0    65         0         0
6         1         1         0    80         80         0
7         1         0         0    40         0         0
8         1         0         0     5         0         0
```

9	1	0	0	35	0	0
10	1	1	0	5	5	0
11	1	0	0	-5	0	0
12	1	1	0	55	55	0
13	1	0	1	60	0	60
14	1	0	1	45	0	45
15	1	0	1	95	0	95

Note that the formula that we use to define our LM contains only all the *known* or *observable* variables, both the outcome variable and the input variables. The *unknown latent* variables—the β s and σ —are not mentioned in the formula. That’s because those are the variables we’re asking to be *estimated*, e.g., one β for each predictor. The estimates that we get from fitting the LM are what is reported in the output of the LM if you print or summarize it.

Working through an example

Next, we’ll go through an example. We’ll use the data from Tan & Jaeger (2025). *Learning to understand an unfamiliar talker: Testing distributional learning as a model of rapid adaptive speech perception*. The data come from an experiment on adaptive speech perception, and contain both responses and reaction times from a two-alternative forced-choice task (2AFC). From the abstract of Tan & Jaeger:

[...] a few minutes of exposure can significantly reduce the processing difficulty listeners experience during initial encounters with an unfamiliar accent. How such adaptation unfolds incrementally, however, remains largely unknown, leaving basic predictions by theories of adaptive speech perception untested. [...] We begin to address these knowledge gaps in a novel incremental exposure-test paradigm. We expose US English listeners to shifted phonetic distributions of word-initial stops (e.g., “dill” vs. “till”), while incrementally assessing cumulative changes in listeners’ perception. [...]

The following figure illustrates the design of the study, consisting of three between-participant exposure conditions, crossed with nine within-participant blocks, each eliciting responses along the phonetic VOT continuum (measured at 12 points). VOT refers to voice onset timing, and is the primary cue to the contrasts like *dip* and *tip* in US English. It is the time between the opening of a speaker’s lips (release the air burst) and the start of vocal fold vibrations. As little as 5ms difference in that time can make the difference between hearing the word *dip* (lower VOT) and the word *tip* (higher VOT). The three

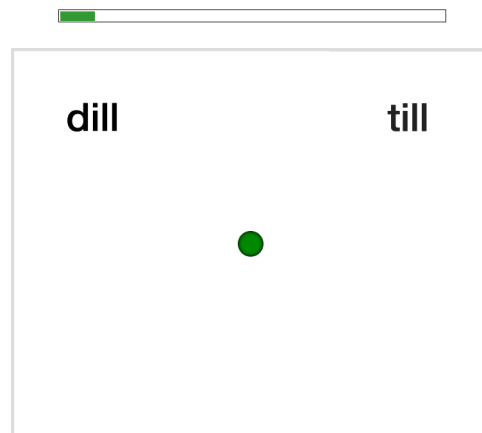


Figure 4: The same simple 2AFC task on all trials. Participants started each trial by clicking the green button. A recording played, and **participants had to answer which of the two words they heard**.

between-participant condition differed only in the distribution of VOT during exposure blocks.

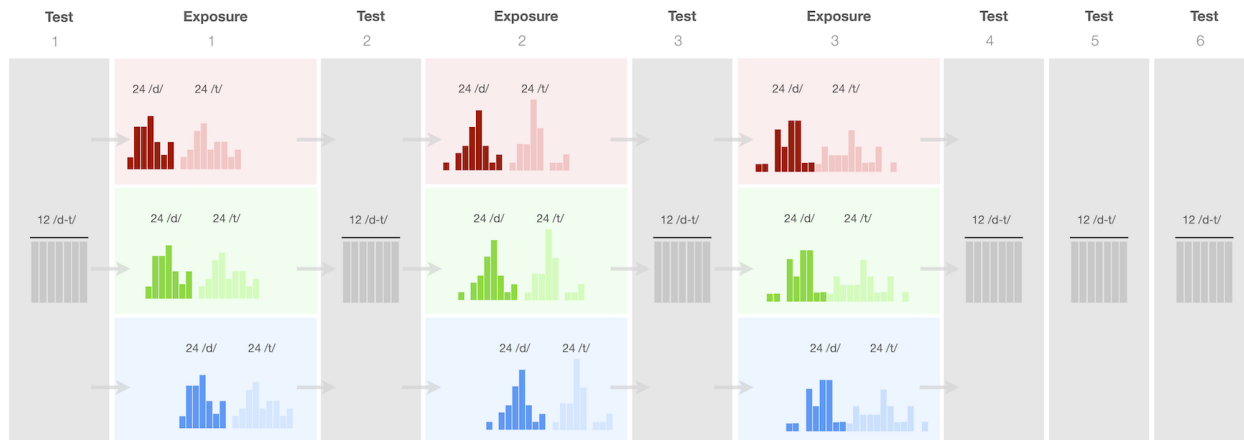


Figure 5: Incremental exposure-test design. Histograms summarize distribution of voice onset time (VOT) in each block (dark bars: “d”-words; light: “t”-words). The three *between-participant* conditions (rows) differed only in the distribution of VOT during exposure blocks. VOTs in test blocks were identical within & across conditions.

Specifically, the green condition has the exact same distribution as the red condition, but shifted 10ms to the right; the blue condition is shifted +40ms to the right. This design makes it possible to assess how participants’ interpretation of the same physical VOT input during the gray test blocks changes as a function of the differently shifted VOT distributions during the exposure blocks. **This makes it possible to test whether listeners change their categorization functions over VOT as a function of recent input from a talker.**

121 participants were included for analysis, each providing 144 exposure trials (3 blocks) and 72 test trials (6 blocks). Participants were unaware of the exposure-test structure of the experiment, and the procedure for exposure and test trials was identical: participants saw two words on the screen (e.g., “dip” and “tip”). Participants then heard a recording of a single word and then had to click the word they heard. All recordings were spoken by the same voice. Here **we use only the data from the six gray test blocks**, leaving 26352 observations.

The data

Since we want to avoid violations of the assumption of independence, we use the data from a single participant from the Shift40 condition,

leaving us with $6 * 12 = 72$ observations. Here's some of the data of that participant. If you downloaded this R markdown document, feel free to change the participant, and see how that changes the results.

	ParticipantID	Condition	Phase	Block	Trial	VOT	Response.isT	Response.RT
1	124	Shift40	test	1	1	50	TRUE	2325
2	124	Shift40	test	1	2	55	TRUE	1791
3	124	Shift40	test	1	3	25	FALSE	1445
4	124	Shift40	test	1	4	30	FALSE	1999
5	124	Shift40	test	1	5	15	FALSE	1453
6	124	Shift40	test	1	6	65	TRUE	1641
7	124	Shift40	test	1	7	40	TRUE	4900
8	124	Shift40	test	1	8	35	TRUE	1489
9	124	Shift40	test	1	9	-5	FALSE	3543
10	124	Shift40	test	1	10	5	FALSE	1587
11	124	Shift40	test	1	11	45	FALSE	5549
12	124	Shift40	test	1	12	70	TRUE	1211
13	124	Shift40	test	3	1	30	FALSE	1337
14	124	Shift40	test	3	2	45	FALSE	1788
15	124	Shift40	test	3	3	50	FALSE	3213

Example 1: Analyzing the effects of VOT on response times

We'll begin by analyzing the effects of VOT on response RTs. This makes response RTs our outcome variable, and VOTs the input variable. The first decision we need to make is how we'd like to code VOT. For example, we could treat VOT as a continuous predictor or as a factor with 12 distinct levels (the number of measurement locations along VOT). And, if we treat VOT as a continuous predictor, we could include only linear VOT or also quadratic VOT or higher order polynomials, provided we have enough data to avoid overfitting.⁶ Similarly, we could decide to include log-transformed VOT instead of VOT, or other transforms. As described above, the linearity assumption of LMs does not prevent us from applying non-linear transform to any of the input variables.

For the present example, we will treat VOT as a continuous predictor and only include its linear component.⁷

A second decision we need to make—not necessarily in this order—is what statistical framework to use. For example, we could use a frequentist framework or we could use a Bayesian framework. If you've never heard about these frameworks, I recommend reading Ch 1 of Gelman et al. 2025 (“Regressions and other stories”). It provides a *very* concise, high-level and neutral overview of these two ways of thinking about statistics. There is a lot of decisions hidden in the choice of framework, and the choice does have consequences for how we fit the model. The choice between these frameworks also *can*,

⁶ As a rule of thumb, the number of coefficients in the LM should be less than $1/20$ th of the number of data points, $k < n/20$. But that's just a rule of thumb, and details depend on how balanced the data is with regard to the distribution of predictor values.

⁷ When we treat a predictor x as continuous, we making the assumption that a change in its value of 1 unit has the same effect on the outcome y , regardless of where along x that change is added. That follows from the linearity assumption mentioned above.

but doesn't necessarily always, affect the conclusions we would draw from our analysis. That is why we will revisit the distinction between frequentist and Bayesian statistics later in the semester.

For now, here's a frequentist LM. The R function `lm` automatically fits to the model with the specified formula to the data `dd`:

```
lm.frequentist <- lm(formula = Response.RT ~ 1 + VOT, data = dd)
print(lm.frequentist)
```

Call:

```
lm(formula = Response.RT ~ 1 + VOT, data = dd)
```

Coefficients:

```
(Intercept)      VOT
  2205.812      -7.595
```

This basic output, which shows the best-fitting parameter estimates for the intercept, $\hat{\alpha}$, and the slope of VOT, $\hat{\beta}$, looks almost identical if we instead look at a Bayesian LM.⁸ Both outputs, also start with a brief summary of what type of model we fit (an LM, which assumes gaussian/normally-distributed residuals) and what formula we used:

```
lm.bayesian <- stan_glm(formula = Response.RT ~ 1 + VOT, data = dd)
print(lm.bayesian)
```

```
stan_glm
family:      gaussian [identity]
formula:     Response.RT ~ 1 + VOT
observations: 72
predictors:  2
-----
              Median MAD_SD
(Intercept) 2207.0  197.6
VOT         -7.7    4.6

Auxiliary parameter(s):
              Median MAD_SD
sigma 860.5   68.6
```

```
-----
* For help interpreting the printed output see ?print.stanreg
* For info on the priors used see ?prior_summary.stanreg
```

The specific output will depend on the package you use. For this particular set of functions, the default output of the Bayesian `stan_glm` function is a bit more verbose; it also prints, for instance, the estimate of the residual standard deviation, $\hat{\sigma}_\epsilon$. This information is also present in the frequentist model; it just wasn't plotted as part of the default output:

⁸ These estimates were obtained using `stan_glm`'s (weakly informative) default priors, which have little influence on the posterior distribution of the parameters, and help avoid overfitting. If that all sounds like gibberish to you, don't fret. We'll cover priors and Bayesian statistics in later lectures.

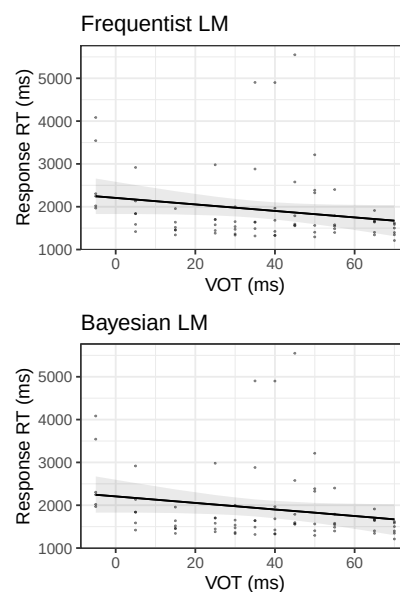


Figure 6: For the present example, the frequentist and Bayesian LM yield similar estimates, and make very similar predictions. The plots show the predicted response RTs as a function of VOT, with 95% confidence intervals (frequentist) or credible intervals. Points show the data.

```
sigma(lm.frequentist)
```

```
[1] 856.6223
```

While the estimates of the two models aren't exactly identical, they are quite similar for this example (see also Figure 6). This won't always be the case. In later lectures, we'll cover how the choice of prior for Bayesian models affects the outcomes, why this is sometimes used to criticize Bayesian statistics, and why this criticism can be misguided. For now, we'll focus on the frequentist LM output, which is likely more familiar. We can obtain more information about the fitted model by summarizing it:

```
summary(lm.frequentist)
```

```
Call:
lm(formula = Response.RT ~ 1 + VOT, data = dd)

Residuals:
    Min       1Q   Median       3Q      Max
-749.9 -451.7 -287.1    2.4 3685.0

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2205.812    191.953   11.491  <2e-16 ***
VOT          -7.595      4.556   -1.667    0.1 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 856.6 on 70 degrees of freedom
Multiple R-squared:  0.03818,    Adjusted R-squared:  0.02444
F-statistic: 2.779 on 1 and 70 DF,  p-value: 0.1
```

This output can be split into four parts. At the top, the model call is repeated, which is convenient, for example, when we look at a stored model object that we created some time ago. Below it, we see a statistical summary of the residuals.⁹ This summary can be helpful in identifying violations of the assumption of normality. Here, for example, there is clear summary that the residuals are *not* normally distributed: the minimum/maximum and quantiles are not symmetrical around zero. This is to be expected—but often ignored—for RTs, which are known to be right-skewed with heavy right tails (due to the boundedness of RTs).¹⁰

The third part of the output summarizes the coefficient estimates ($\hat{\beta}$), their standard errors, the resulting t -statistic, and its p -value. The t -statistics is a signed measure of the estimates distance from 0 in

⁹ Recall that *residuals* are the prediction error of the LM, the difference between the actual and the predicted outcome. Each row of the model matrix X corresponds to a prediction for that row, and thus has a corresponding residual ϵ .

¹⁰ For review and large-scale statistical simulations that assess the consequences of using LMs and its repeated-measures extension LMMs for the analysis of RTs, see Burchill & Jaeger (2024) in *JML*.

terms of its own standard error ($t = \frac{\hat{\beta}}{SE_{\hat{\beta}}}$).¹¹ The p -value is the result of a Student’s t -test with the residual DFs of the LM as DFs for the test. E.g., for the effect of VOT, $t(70) = -1.667$. And if we look the probability density function for the Student t distribution to ask how probable this t -value is for $DF = 70$, we’ll get 0.1, matching the p -value reported in the LM for the effect of VOT.

Finally, the bottom part of the LM output provides multiple measures of the goodness of fit of the LM. Of these, the R^2 tells us how much of the variance in the outcome y is accounted for by the model (*for this sample*). Notably, an LM can have one or more statistically significant effects even if the model overall accounts for very little variance in out observed outcome. The F -statistic (here $F(1, 70) = 2.779$) tests whether the predictors included in the model *together* provide for a statistically significantly better fit, compared to an LM with just the intercept (the “null model”). The DFs for this test will always be the number of predictors in the model. We will return to this F -test when we talk about model comparison.

Write-up

For this particular example, there isn’t much to report. Note that we wouldn’t usually care about the intercept, and rarely report it. So we might write:

To test our hypothesis, we used linear regression to regress the participant’s reaction time against VOT. The effect of VOT was not significant ($\hat{\beta} = -7.6$, $t = -1.67$, $p \geq .1$).

A few considerations:

- I recommend you provide both the relevant statistics (coefficient estimates, p -values, etc.), and a descriptive interpretation of that result in non-technical language—but be careful to avoid language that is wrong.
- **Use $\hat{\beta}$** when we describe coefficient estimates and write up results. This emphasizes that these are *estimates* of the assumed population parameters (β).
- Try to resist language that suggests significance even when it was not found, as in “... did not reach significance”.
- Do not take p -values to indicate effect strength. It is ok to write things like “highly significant effect” but that is not the same as a “large effect”. Similarly, a “large effect”—in whatever way that is determined—is not necessarily the same as a “theoretically particularly important” effect.

¹¹ This is just the [standard definition of a \$t\$ -value](#) as a measure of how different an estimate is from some target value. Here, the target value is 0 since we want to know whether the coefficient for a predictor is different from 0. If it is, we say that the predictor “has an effect” on the outcome.

Questions to ponder in class

- How would you interpret the above output?
- How would we expect the output to change if we centered VOT by subtracting the mean VOT from each VOT value before entering it into the model? Which estimates of the model would change? Which ones would stay the same?
- How about if we also standardized VOT (by dividing it by its standard deviation)?
- How could we test whether the slope of VOT changes across Blocks?

Bonus question:

- How could we test a somewhat more interesting hypothesis that more ambiguous VOTs—VOTs that are more likely to elicit an equal proportion of “d”- and “t”-responses—are responded to more slowly?

A Bayesian LM

Finally, let's return to the Bayesian LM we fit above. We can summarize it in the same way as we did for the frequentist LM:

```
summary(lm.bayesian)
```

Model Info:

```
function:      stan_glm
family:        gaussian [identity]
formula:       Response.RT ~ 1 + VOT
algorithm:     sampling
sample:        4000 (posterior sample size)
priors:        see help('prior_summary')
observations:  72
predictors:    2
```

Estimates:

	mean	sd	10%	50%	90%
(Intercept)	2206.4	194.9	1954.1	2207.0	2452.8
VOT	-7.7	4.6	-13.6	-7.7	-1.7
sigma	865.9	71.3	779.8	860.5	957.9

Fit Diagnostics:

	mean	sd	10%	50%	90%
mean_ppd	1934.0	144.0	1754.0	1934.2	2117.0

The mean_ppd is the sample average posterior predictive distribution of the outcome variable (for details see `help('summary.stan')`)

MCMC diagnostics

```
mcse Rhat n_eff
```

```
(Intercept)  3.1  1.0  3897
VOT          0.1  1.0  4012
sigma        1.2  1.0  3446
mean_PPD     2.4  1.0  3702
log-posterior 0.0  1.0  1734
```

For each parameter, `mcse` is Monte Carlo standard error, `n_eff` is a crude measure of effective sample size, and `Rhat` is the potential

Again, the first part summarizes the model we fit, the data we fit it to, and some information about how the model was fit. The second part provides estimates of the coefficients, and their standard errors (= the standard deviation of the estimates). Unlike the frequentist model, the Bayesian LM does not provide a t -statistic or a p -value based on that statistic. In fact, we don't need either of these parametric summaries of the uncertainty we should maintain about the estimate. That's because the model represents this uncertainty in the 4000 posterior samples it has drawn from the posterior distribution of the three parameters of the LM. The quantiles of these posterior samples are summarized in the output of the LM.¹² These are the posterior samples that we can use to test hypotheses about the parameters of the model, e.g., by asking how many of the posterior samples are above or below 0. For example, we can ask whether RTs decrease with increasing VOT, i.e., whether the coefficient for VOT is negative:

```
hypothesis(lm.bayesian, "VOT < 0")
```

```
Hypothesis Tests for class :
Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio Post.Prob Star
1 (VOT) < 0      -7.66      4.63   -15.27    0.02      18.8      0.95
---
'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
'*': For one-sided hypotheses, the posterior probability exceeds 95%;
for two-sided hypotheses, the value tested against lies outside the 95%-
CI.
Posterior probabilities of point hypotheses assume equal prior probabilities.
```

¹² Note one interesting difference between the Bayesian and frequentist LM that we'll return to in later lectures: in the frequentist LM, estimate of the standard deviation of the residuals $\hat{\sigma}_\epsilon$ is a point estimate—when we obtained `sigma(lm.frequentist)` above, there was no uncertainty associated with that estimate, unlike for the estimates of the intercept and slope of VOT. This is different in the Bayesian LM, in which we obtain the full posterior distribution of σ_ϵ (summarized in terms of its quantiles in the output). Bayesians maintain uncertainty about *all* parameters in the model.

In this case, the hypothesis test tells us that the mean estimate is indeed negative and that 95% of the posterior samples are below 0. We also see an evidence ratio—essentially a Bayes Factor, about which we'll learn more later. This evidence ratio tells us the odds that a random posterior sample of the 4000 samples in the fitted model supports the hypothesis we tested. For the present example, these odds are roughly 17.9-to-1. Based on common conventions, we might summarize that the evidence for a negative effect is “moderate”, though some Bayesians would prefer not to use *any* dichotomizing language like that.

The final part of the Bayesian LM summary contains a number of diagnostics. We won't cover them for now, but the fact that the

Rhat's are all very close to 1 indicates that the model has converged and that we have no evidence that the posterior samples contained in the fitted model don't reflect the posterior distribution.

Session info

```
devtools::session_info()
```

```
- Session info -----
setting  value
version  R version 4.5.2 (2025-10-31)
os       macOS Sonoma 14.5
system   aarch64, darwin20
ui       X11
language (EN)
collate  en_US.UTF-8
ctype    en_US.UTF-8
tz       America/New_York
date     2026-09-06
pandoc   3.10 @ /opt/homebrew/bin/ (via rmarkdown)
quarto   1.10.18 @ /usr/local/bin/quarto

- Packages -----
package      * version    date (UTC) lib source
abind         1.4-8      2024-09-12 [1] CRAN (R 4.5.0)
backports     1.5.1      2026-04-03 [1] CRAN (R 4.5.2)
base64enc     0.1-6      2026-02-02 [1] CRAN (R 4.5.2)
bayesplot     1.15.0     2025-12-12 [1] CRAN (R 4.5.2)
boot         1.3-32     2025-08-29 [1] CRAN (R 4.5.2)
bridgesampling 1.2-1      2025-11-19 [1] CRAN (R 4.5.2)
brms          * 2.23.0     2025-09-09 [1] CRAN (R 4.5.0)
Brodbingnag   1.2-9      2022-10-19 [1] CRAN (R 4.5.0)
broom        * 1.0.13     2026-05-14 [1] CRAN (R 4.5.2)
cachem        1.1.0      2024-05-16 [1] CRAN (R 4.5.0)
checkmate     2.3.4      2026-02-03 [1] CRAN (R 4.5.2)
cli           3.6.6      2026-04-09 [1] CRAN (R 4.5.2)
coda          0.19-4.1   2024-01-31 [1] CRAN (R 4.5.0)
codetools     0.2-20     2024-03-31 [1] CRAN (R 4.5.2)
collapse      2.1.7      2026-05-19 [1] CRAN (R 4.5.2)
colorspace    2.1-3      2026-07-12 [1] CRAN (R 4.5.2)
colourpicker  1.3.0      2023-08-21 [1] CRAN (R 4.5.0)
crayon        1.5.3      2024-06-20 [1] CRAN (R 4.5.0)
crosstalk     1.2.2      2025-08-26 [1] CRAN (R 4.5.0)
curl          7.1.0      2026-04-22 [1] CRAN (R 4.5.2)
data.table    1.18.6.1   2026-08-24 [1] CRAN (R 4.5.2)
devtools      2.5.2      2026-04-30 [1] CRAN (R 4.5.2)
digest        0.6.39     2025-11-19 [1] CRAN (R 4.5.2)
distributional 0.8.1      2026-06-27 [1] CRAN (R 4.5.2)
dplyr         * 1.2.1      2026-04-03 [1] CRAN (R 4.5.2)
DT            0.34.0     2025-09-02 [1] CRAN (R 4.5.0)
dygraphs      1.1.1.6    2018-07-11 [1] CRAN (R 4.5.0)
ellipsis      0.3.3      2026-04-04 [1] CRAN (R 4.5.2)
emmeans       2.0.4      2026-07-15 [1] CRAN (R 4.5.2)
```

estimability	2.0.0	2026-06-26	[1]	CRAN	(R 4.5.2)
evaluate	1.0.5	2025-08-27	[1]	CRAN	(R 4.5.0)
farver	2.1.2	2024-05-13	[1]	CRAN	(R 4.5.0)
fastmap	1.2.0	2024-05-15	[1]	CRAN	(R 4.5.0)
forcats	* 1.0.1	2025-09-25	[1]	CRAN	(R 4.5.0)
fs	2.1.0	2026-04-18	[1]	CRAN	(R 4.5.2)
generics	0.1.4	2025-05-09	[1]	CRAN	(R 4.5.0)
gganimate	* 1.0.11	2025-09-04	[1]	CRAN	(R 4.5.0)
ggplot2	* 4.0.3	2026-04-22	[1]	CRAN	(R 4.5.2)
ggrepel	* 0.9.8	2026-03-17	[1]	CRAN	(R 4.5.2)
ggthemes	* 5.2.0	2025-11-30	[1]	CRAN	(R 4.5.2)
gifski	1.32.0-2	2025-03-18	[1]	CRAN	(R 4.5.0)
glue	1.8.1	2026-04-17	[1]	CRAN	(R 4.5.2)
gridExtra	2.3.1	2026-06-25	[1]	CRAN	(R 4.5.2)
gtable	0.3.6	2024-10-25	[1]	CRAN	(R 4.5.0)
gtools	3.9.5	2023-11-20	[1]	CRAN	(R 4.5.0)
hms	1.1.4	2025-10-17	[1]	CRAN	(R 4.5.0)
htmltools	0.5.9	2025-12-04	[1]	CRAN	(R 4.5.2)
htmlwidgets	1.6.4	2023-12-06	[1]	CRAN	(R 4.5.0)
httpuv	1.6.17	2026-03-18	[1]	CRAN	(R 4.5.2)
igraph	2.3.3	2026-06-26	[1]	CRAN	(R 4.5.2)
inline	0.3.21	2025-01-09	[1]	CRAN	(R 4.5.0)
insight	1.5.2	2026-06-28	[1]	CRAN	(R 4.5.2)
jsonlite	2.0.0	2025-03-27	[1]	CRAN	(R 4.5.0)
knitr	* 1.51	2025-12-20	[1]	CRAN	(R 4.5.2)
labeling	0.4.3	2023-08-29	[1]	CRAN	(R 4.5.0)
later	1.4.8	2026-03-05	[1]	CRAN	(R 4.5.2)
lattice	0.23-1	2026-08-12	[1]	CRAN	(R 4.5.2)
lifecycle	1.0.5	2026-01-08	[1]	CRAN	(R 4.5.2)
lme4	2.0-6	2026-07-16	[1]	CRAN	(R 4.5.2)
loo	2.10.1	2026-07-24	[1]	CRAN	(R 4.5.2)
lubridate	* 1.9.5	2026-02-04	[1]	CRAN	(R 4.5.2)
magrittr	* 2.0.5	2026-04-04	[1]	CRAN	(R 4.5.2)
marginaleffects	* 0.32.0	2026-02-14	[1]	CRAN	(R 4.5.2)
markdown	2.0	2025-03-23	[1]	CRAN	(R 4.5.0)
MASS	* 7.3-66	2026-07-15	[1]	CRAN	(R 4.5.2)
Matrix	1.7-6	2026-07-25	[1]	CRAN	(R 4.5.2)
matrixStats	1.5.0	2025-01-07	[1]	CRAN	(R 4.5.0)
memoise	2.0.1	2021-11-26	[1]	CRAN	(R 4.5.0)
mime	0.13	2025-03-17	[1]	CRAN	(R 4.5.0)
miniUI	0.1.2	2025-04-17	[1]	CRAN	(R 4.5.0)
minqa	1.2.8	2024-08-17	[1]	CRAN	(R 4.5.0)
multcomp	1.4-31	2026-07-12	[1]	CRAN	(R 4.5.2)
mvtnorm	1.4-2	2026-07-12	[1]	CRAN	(R 4.5.2)
nlme	3.1-170	2026-07-15	[1]	CRAN	(R 4.5.2)
nloptr	2.2.1	2025-03-17	[1]	CRAN	(R 4.5.0)
otel	0.2.0	2025-08-29	[1]	CRAN	(R 4.5.0)
patchwork	* 1.3.2	2025-08-25	[1]	CRAN	(R 4.5.0)
pillar	1.11.1	2025-09-17	[1]	CRAN	(R 4.5.0)
pkgbuild	1.4.8	2025-05-26	[1]	CRAN	(R 4.5.0)
pkgconfig	2.0.3	2019-09-22	[1]	CRAN	(R 4.5.0)
pkgload	1.5.3	2026-06-15	[1]	CRAN	(R 4.5.2)
plyr	1.8.9	2023-10-02	[1]	CRAN	(R 4.5.0)
png	0.1-9	2026-03-15	[1]	CRAN	(R 4.5.2)
posterior	1.7.0	2026-04-01	[1]	CRAN	(R 4.5.2)
prettyunits	1.2.0	2023-09-24	[1]	CRAN	(R 4.5.0)
progress	1.2.3	2023-12-06	[1]	CRAN	(R 4.5.0)
progressr	1.0.0	2026-07-04	[1]	CRAN	(R 4.5.2)

promises	1.5.0	2025-11-01	[1]	CRAN	(R 4.5.0)
purrr	* 1.2.2	2026-04-10	[1]	CRAN	(R 4.5.2)
QuickJSR	1.11.0	2026-08-21	[1]	CRAN	(R 4.5.2)
R6	2.6.1	2025-02-15	[1]	CRAN	(R 4.5.0)
rbibutils	2.4.1	2026-01-21	[1]	CRAN	(R 4.5.2)
RColorBrewer	1.1-3	2022-04-03	[1]	CRAN	(R 4.5.0)
Rcpp	* 1.1.2	2026-07-05	[1]	CRAN	(R 4.5.2)
RcppParallel	6.2.1	2026-08-27	[1]	CRAN	(R 4.5.2)
Rdpack	2.6.6	2026-02-08	[1]	CRAN	(R 4.5.2)
readr	* 2.2.0	2026-02-19	[1]	CRAN	(R 4.5.2)
reformulas	0.4.4	2026-02-02	[1]	CRAN	(R 4.5.2)
reshape2	1.4.5	2025-11-12	[1]	CRAN	(R 4.5.0)
rlang	1.3.0	2026-07-05	[1]	CRAN	(R 4.5.2)
rmarkdown	2.31	2026-03-26	[1]	CRAN	(R 4.5.2)
rstan	2.32.7	2025-03-10	[1]	CRAN	(R 4.5.0)
rstanarm	* 2.32.2	2025-09-30	[1]	CRAN	(R 4.5.0)
rstantools	2.7.1	2026-08-29	[1]	CRAN	(R 4.5.2)
S7	0.2.2	2026-04-22	[1]	CRAN	(R 4.5.2)
sandwich	3.1-3	2026-08-03	[1]	CRAN	(R 4.5.2)
scales	1.4.0	2025-04-24	[1]	CRAN	(R 4.5.0)
scatterplot3d	* 0.3-45	2026-02-23	[1]	CRAN	(R 4.5.2)
sessioninfo	1.2.4	2026-06-04	[1]	CRAN	(R 4.5.2)
shiny	1.14.0	2026-06-21	[1]	CRAN	(R 4.5.2)
shinyjs	2.1.1	2026-01-15	[1]	CRAN	(R 4.5.2)
shinytan	2.7.0	2025-12-12	[1]	CRAN	(R 4.5.2)
shinythemes	1.2.0	2021-01-25	[1]	CRAN	(R 4.5.0)
StanHeaders	2.32.10	2024-07-15	[1]	CRAN	(R 4.5.0)
stringi	1.8.9	2026-08-04	[1]	CRAN	(R 4.5.2)
stringr	* 1.6.0	2025-11-04	[1]	CRAN	(R 4.5.0)
survival	3.8-9	2026-07-08	[1]	CRAN	(R 4.5.2)
tensorA	0.36.2.1	2023-12-13	[1]	CRAN	(R 4.5.0)
TH.data	1.1-5	2025-11-17	[1]	CRAN	(R 4.5.2)
threejs	0.3.4	2025-04-21	[1]	CRAN	(R 4.5.0)
tibble	* 3.3.1	2026-01-11	[1]	CRAN	(R 4.5.2)
tidyr	* 1.3.2	2025-12-19	[1]	CRAN	(R 4.5.2)
tidyselect	1.2.1	2024-03-11	[1]	CRAN	(R 4.5.0)
tidyverse	* 2.0.0	2023-02-22	[1]	CRAN	(R 4.5.0)
timechange	0.4.0	2026-01-29	[1]	CRAN	(R 4.5.2)
tinytex	0.60	2026-06-16	[1]	CRAN	(R 4.5.2)
tufte	* 0.15.0	2026-05-21	[1]	CRAN	(R 4.5.2)
tweenr	2.0.3	2024-02-26	[1]	CRAN	(R 4.5.0)
tzdb	0.5.0	2025-03-15	[1]	CRAN	(R 4.5.0)
usethis	3.2.1	2025-09-06	[1]	CRAN	(R 4.5.0)
V8	8.2.0	2026-04-21	[1]	CRAN	(R 4.5.2)
vctrs	0.7.3	2026-04-11	[1]	CRAN	(R 4.5.2)
withr	3.0.3	2026-06-19	[1]	CRAN	(R 4.5.2)
xfun	0.60	2026-07-09	[1]	CRAN	(R 4.5.2)
xtable	1.8-8	2026-02-22	[1]	CRAN	(R 4.5.2)
xts	0.14.2	2026-02-28	[1]	CRAN	(R 4.5.2)
yaml	2.3.12	2025-12-10	[1]	CRAN	(R 4.5.2)
zoo	1.9-0	2026-07-31	[1]	CRAN	(R 4.5.2)

[1] /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library

* -- Packages attached to the search path.